AlgoWeaver：LLM Agent 笔试要求对照报告

功能核对、实现证据、评分自评与交付结论

1. 项目摘要

AlgoWeaver 是一个算法能力工厂原型。系统将业务资料、历史代码、实验记录和验证结果组织成结构化能力资产，并通过检索、规划、代码生成、自动验证、有限修复和版本化复用，形成从能力描述到可运行算法代码的闭环。

主演示场景是库存需求预测，补充场景包括异常检测和预测性维护。项目提供 Python CLI、FastAPI API 和 Streamlit 可视化工作台。

2. 需求逐条核对

状态说明：

- **已实现**：代码、测试或运行证据已确认。

- **部分实现**：已有可运行路径，但覆盖范围、现场条件或证据仍有限。

- **待扩展**：当前版本没有完整实现。

2.1 基础功能要求

/ 要求 / 状态 / 项目对应实现与证据 /

/ / / /

/ LLM 理解、知识抽取、代码生成或修复 / 部分实现 / `llm/` provider 接口、LiteLLM 适配器、Mock 模式、Pydantic 输出校验、代码生成与修复边界；真实 provider 需要在目标环境配置后复核 /

/ 具体行业场景 / 已实现 / `examples/inventory_forecast/`、`examples/anomaly_detection/`、`examples/predictive_maintenance/` /

/ 行业能力知识库/知识图谱 / 部分实现 / Pydantic 能力卡片、SQLite 版本库、JSON-LD 导出、证据节点和失败经验；完整图谱可视化仍待扩展 /

/ 自然语言到可运行代码 / 部分实现 / `reproduce` 流程、模板/Mock/LLM 生成器、Python artifact 与测试生成；通用任务覆盖仍有限 /

/ 统一验证机制 / 部分实现 / AST、接口契约、单元测试、运行报告、MAE/RMSE、分类指标、Docker/WSL 隔离和状态门禁 /

/ UI、CLI 或 API / 已实现 / Typer CLI、FastAPI、Streamlit 工作台；工作台包含任务描述、资料上传、候选比较和验证结果展示 /

2.2 a-g 核心闭环

/ 阶段 / 状态 / 证据 /

/ / / /

/ a. 理解能力描述 / 已实现 / `CapabilityExtractor`、Mock/LLM 模式、Pydantic schema、来源证据字段 /

/ b. 检索相关知识 / 部分实现 / SourceIngestor、chunker、词法检索、可选 Qdrant、embedding 和 reranker /

/ c. 规划实现方案 / 已实现 / LangGraph reproduction graph、ReproductionPlanner、结构化 plan /

/ d. 生成可运行代码 / 部分实现 / TemplateArtifactWriter、MockCodeGenerator、LLMCodeGenerator；生成代码默认保持 unverified /

/ e. 自动测试与指标 / 部分实现 / Python 测试、AST、运行报告、MAE/RMSE、分类指标和资源限制；WSL/Docker 取决于目标环境 /

/ f. 错误修复与优化 / 部分实现 / BoundedRepairLoop、TemplateRepairer、LLMRepairer、失败观察记录；多轮修复覆盖仍有限 /

/ g. 回写知识库 / 部分实现 / SQLite capability registry、verified gate、JSON-LD、RepairFailureRecord；图谱可视化尚未提供 /

3. 开发要求核对

- Git 版本控制：已实现，默认分支和功能分支均有清晰提交记录。

- 项目文档：已实现，包含 README、架构说明、运行方式、示例和公开交付报告。

- 模块化：已实现，按 agents、schemas、services、llm、examples、tests 分层。

- 本地复现：部分实现，Mock/模板路径可离线复现；真实 provider、WSL 和外部服务需配置。

- Python：已实现，Python 3.12/3.13 CI 通过。

- 框架选择：Pydantic、LangGraph、SQLite、FastAPI、Typer、Streamlit、Qdrant、Docker/WSL 均有对应代码边界。

- 无法部署大模型时的替代：已实现，提供 Mock LLM 和模板路径。

4. 交付物核对

/ 交付项 / 状态 / 位置或说明 /

/ / / /

/ 完整代码库 / 已实现 / GitHub 默认分支 /

/ 示例数据与材料 / 已实现 / `examples/` /

/ 能力 schema / 已实现 / `src/algo_weaver/schemas/` /

/ 知识图谱/导出结果 / 部分实现 / JSON-LD 导出；未提供完整图形可视化 /

/ Agent 工作流 / 已实现 / `src/algo_weaver/agents/` /

/ 端到端示例 / 已实现 / inventory forecast workflow /

/ 自动验证脚本 / 已实现 / pytest、CLI、API 和 validation services /

/ 验证报告样例 / 已实现 / `docs/reports/` 和 PDF 交付报告 /

/ README / 已实现 / 面向导师的公开版 README /

/ 依赖文件 / 已实现 / `pyproject.toml`、`uv.lock` /

/ 使用示例与测试 / 已实现 / `examples/`、`tests/` /

5. 进阶要求核对

/ 进阶项 / 状态 / 说明 /

/ / / /

/ Web/CLI/API / 已实现 / Streamlit、Typer、FastAPI /

/ 多轮代码修复 / 部分实现 / 有界修复和修复记录，覆盖范围仍有限 /

/ 多候选算法比较 / 已实现 / 三种库存预测候选、MAE/RMSE 门禁、JSON/Markdown 报告、API/UI 入口 /

/ 知识图谱可视化 / 待扩展 / 当前提供 JSON-LD 导出和静态验收图，不是交互式图谱浏览器 /

/ 自动生成验证报告 / 部分实现 / 运行报告、指标报告和候选比较报告已具备 /

/ 验证结果回写 / 部分实现 / SQLite verified gate、JSON-LD 和失败经验已具备 /

/ 多任务验证配置 / 部分实现 / 回归指标、分类指标、接口和运行验证已具备，任务类型仍需继续扩展 /

/ 插件式模板/指标 / 待扩展 / provider 接口可替换，但尚未形成独立插件注册机制 /

6. 加分项核对

- 图搜索、Beam Search、MCTS：待扩展，目前使用检索排序和候选比较。
- 多智能体协作：待扩展，目前为单一工作流图，不是多个自治 Agent 协作。
- 真实代码仓库抽取：部分实现，已有 AST parser，但仓库级端到端抽取仍需加强。
- 安全检查与沙箱：部分至已实现，Docker/WSL 有只读、禁网、资源和超时边界；现场运行依赖环境。
- 失败案例沉淀：已实现基础版本，RepairFailureRecord 写入 SQLite 并支持 JSON-LD 导出。
- 能力版本管理：已实现，SQLite immutable versions、verified gate 和版本 diff。
- 方案设计解释：部分实现，证据引用、来源定位和验证检查可解释。
- 跨场景迁移：部分实现，已有库存、异常、预测性维护三个示例。
- 接口文档/部署配置：部分实现，FastAPI、OpenAPI、Docker/WSL 和部署文档边界已提供。
- 性能与资源分析：部分实现，已有运行报告和资源边界，但缺少系统化 benchmark。

7. 评分标准自评

技术能力（40%）

核心闭环已经具备可运行原型：能力抽取、检索、规划、生成、验证、有限修复和回写均有对应模块。主要扣分风险是通用任务覆盖、图谱可视化、多 Agent 和真实 provider 的环境依赖。

代码质量（30%）

模块分层清晰，Pydantic 约束明确，测试和静态检查齐全。当前 CI 在 Python 3.12/3.13
上通过；仍应继续补充复杂文档、更多异常场景和性能测试。

创新性（15%）

项目将来源证据、能力状态、验证门禁、候选算法比较和失败经验结合起来，形成“ 验证后再复用 ”的能力资产机制。图搜索、多 Agent 和插件机制是后续创新方向。

完整性（15%）

代码、示例、UI、测试、报告、截图和 PDF
均已提供。正式公网部署、交互式知识图谱和视频录制仍可作为后续展示增强。

8. 结论

AlgoWeaver 已达到“ 可运行算法能力工厂原型 ”的交付水平，能够展示从资料接入、能力理解、检索、规划、代码生成、验证到能力沉淀的主流程。对笔试要求而言，最有说服力的证据是：

1. 可运行的库存预测主场景；
2. CLI、FastAPI 和 Streamlit 三种入口；
3. Pydantic 能力卡片和 SQLite verified gate；
4. 多候选算法比较与指标门禁；

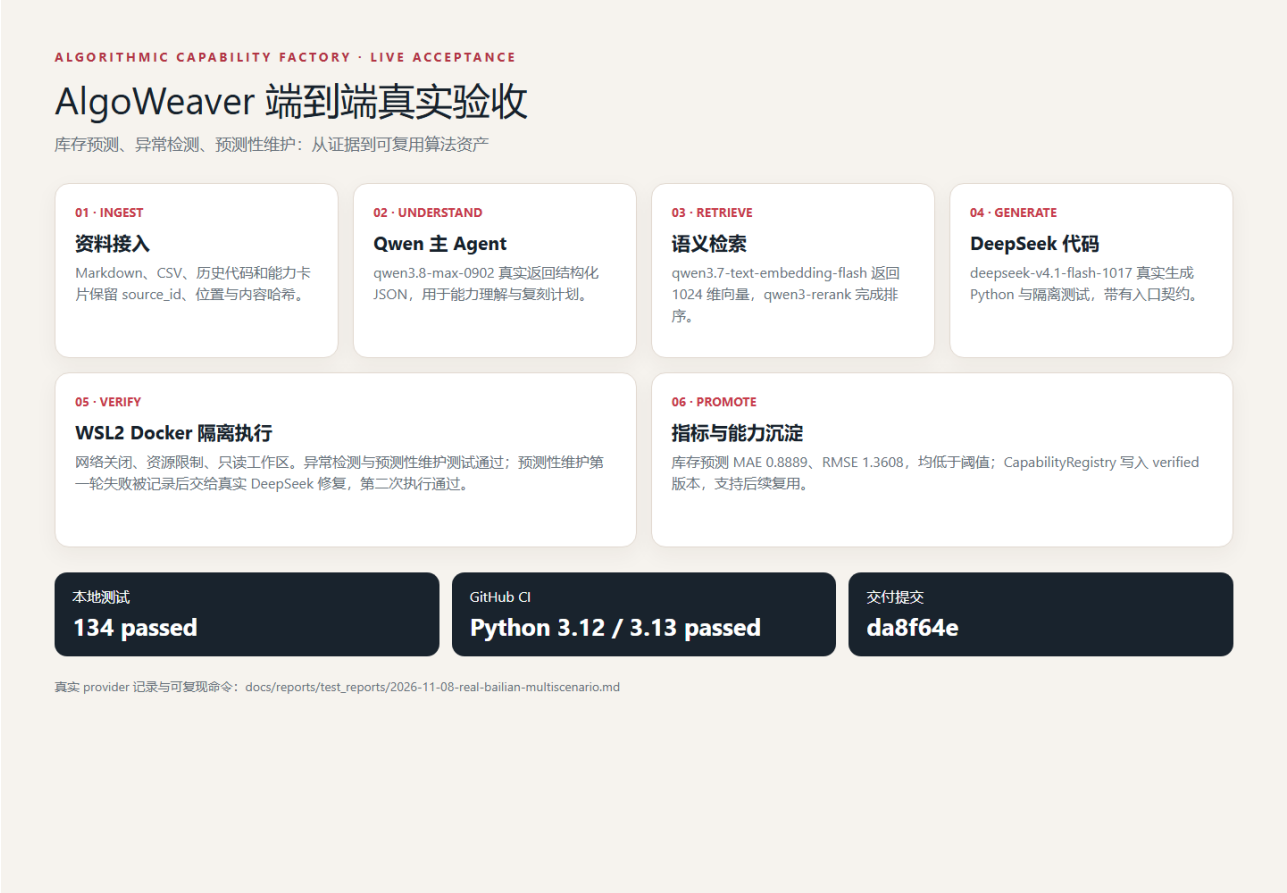
5. 146 passed 的本地验证结果和 Python 3.12/3.13 CI；

6. 可直接提交的 PDF 报告和可视化验收截图。

剩余工作主要属于增强项：交互式图谱、多 Agent、插件注册、多轮修复扩展、性能 benchmark 和正式公网部署。

可视化验收截图

下图来自仓库中的静态验收展示图，展示资料接入、能力理解、检索、生成、验证和能力沉淀六个阶段。



图：AlgoWeaver 端到端验收流程。